

School of Computer Science Engineering and Information Systems

Department of Smart Computing

Bachelor of Computer Applications Winter Semester 2023-2024

DSA – Cycle sheet - 3

Name : Mohammed Azhan Palli

RegNo : 23BCA0288 .

1. Write a C Program to perform Bubble Sort.

Program :

```
#include <stdio.h>
```

```
void bubbleSort(int array[], int size) {  
    // loop to access each array element  
    for (int step = 0; step < size - 1; ++step) {  
        for (int i = 0; i < size - step - 1; ++i) {  
            // compare two adjacent elements  
            if (array[i] > array[i + 1]) {  
                int temp = array[i];  
                array[i] = array[i + 1];  
                array[i + 1] = temp;  
            }  
        }  
    }  
}  
  
// print sorted arrays !
```

```
void printSArray(int array[], int size) {  
    for (int i = 0; i < size; ++i) {  
        printf("%d ", array[i]);  
    }  
    printf("\n");  
}
```

```

int main() {
    int data[] = {30, -23, 11, 45, 17, -21, 100};
    // find the array's length
    int size = sizeof(data) / sizeof(data[0]);
    bubbleSort(data, size);
    printf("Sorted Array in Ascending Order:\n");
    printSArray(data, size);
}

```

Output :

```

C:\Users\blogw\OneDrive\De
Sorted Array in Ascending Order:
-23 -21 11 17 30 45 100
Process returned 0 (0x0) execution time : 0.904 s
Press any key to continue.

```

2. Write a C Program to perform Selection Sort.

Program :

```
#include <stdio.h>
```

```

void selectionSort(int arr[], int n) {
    int i, j, minIndex, temp;
    for (i = 0; i < n - 1; i++) {
        minIndex = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex])

```

```

        minIndex = j;
    }
    if (minIndex != i) {
        temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}
}

```

```

int main() {
    int arr[] = {64, 25, 12, 55, 9, 18, 22, 100};
    int n = sizeof(arr) / sizeof(arr[0]);
    selectionSort(arr, n);
    printf("Sorted array: \n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    return 0;
}

```

Program :

```
"C:\Users\blogw\OneDrive\Di x + v - 0 x
Sorted array:
9 12 18 22 25 55 64 100
Process returned 0 (0x0) execution time : 1.282 s
Press any key to continue.
```

3. Write a C Program to perform Insertion Sort.

Program :

```
#include <stdio.h>
```

```
void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

```
int main() {
    int arr[] = {12, 11, 13, 17, 22, 55, 5, 6};
```

```

int n = sizeof(arr) / sizeof(arr[0]);

insertionSort(arr, n);

printf("Sorted array: \n");

for (int i = 0; i < n; i++)

    printf("%d ", arr[i]);

printf("\n");

return 0;

}

```

Output :

```

C:\Users\blogw\OneDrive\De
Sorted array:
5 6 11 12 13 17 22 55
Process returned 0 (0x0) execution time : 1.231 s
Press any key to continue.

```

4. Write a C Program to perform Shell Sort.

Program :

```
#include <stdio.h>
```

```

void shellSort(int arr[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap)

```

```

        arr[j] = arr[j - gap];

        arr[j] = temp;
    }

}

}

int main() {

    int arr[] = {12, 34, 77, 82, -5, 54, 2, 3};

    int n = sizeof(arr) / sizeof(arr[0]);

    shellSort(arr, n);

    printf("Sorted array: \n");

    for (int i = 0; i < n; i++)

        printf("%d ", arr[i]);

    printf("\n");

    return 0;

}

```

Output :

```

C:\Users\blogw\OneDrive\De
Sorted array:
-5 2 3 12 34 54 77 82
Process returned 0 (0x0)   execution time : 1.135 s
Press any key to continue.

```

5. Write a C Program to perform Quick Sort.

Program :

```
#include <stdio.h>
```

```
void swap(int* a, int* b) {
```

```
    int t = *a;
```

```
    *a = *b;
```

```
    *b = t;
```

```
}
```

```
int partition(int arr[], int low, int high) {
```

```
    int pivot = arr[high];
```

```
    int i = (low - 1);
```

```
    for (int j = low; j <= high - 1; j++) {
```

```
        if (arr[j] < pivot) {
```

```
            i++;
```

```
            swap(&arr[i], &arr[j]);
```

```
        }
```

```
    }
```

```
    swap(&arr[i + 1], &arr[high]);
```

```
    return (i + 1);
```

```
}
```

```
void quickSort(int arr[], int low, int high) {
```

```
    if (low < high) {
```

```
        int pi = partition(arr, low, high);
```

```
        quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
```

```
    }
```

```
}
```

```
int main() {
```

```
    int arr[] = {10, 7, 8, 12, 22, 77, 3, 4, 9, 1, 5};
```

```

int n = sizeof(arr) / sizeof(arr[0]);

quickSort(arr, 0, n - 1);

printf("Sorted array: \n");

for (int i = 0; i < n; i++)

    printf("%d ", arr[i]);

printf("\n");

return 0;

}

```

Output :

```

C:\Users\blogw\OneDrive\De
Sorted array:
1 3 4 5 7 8 9 10 12 22 77
Process returned 0 (0x0) execution time : 1.109 s
Press any key to continue.

```

6. Write a C Program to perform Merge Sort.

Program :

```
#include <stdio.h>
```

```
void merge(int arr[], int l, int m, int r) {
```

```
    int i, j, k;
```

```
    int n1 = m - l + 1;
```

```
    int n2 = r - m;
```

```
    int L[n1], R[n2];
```



```
for (i = 0; i < n1; i++)  
    L[i] = arr[l + i];  
for (j = 0; j < n2; j++)  
    R[j] = arr[m + 1 + j];
```

```
i = 0;  
j = 0;  
k = l;  
while (i < n1 && j < n2) {  
    if (L[i] <= R[j]) {  
        arr[k] = L[i];  
        i++;  
    } else {  
        arr[k] = R[j];  
        j++;  
    }  
    k++;  
}
```

```
while (i < n1) {  
    arr[k] = L[i];  
    i++;  
    k++;  
}
```

```
while (j < n2) {  
    arr[k] = R[j];  
    j++;  
    k++;  
}
```

```
}
```

```
void mergeSort(int arr[], int l, int r) {
```

```
    if (l < r) {
```

```
        int m = l + (r - l) / 2;
```

```
        mergeSort(arr, l, m);
```

```
        mergeSort(arr, m + 1, r);
```

```
        merge(arr, l, m, r);
```

```
    }
```

```
}
```

```
int main() {
```

```
    int arr[] = {12, 11, 13, 100, 200, -4, -11, 5, 6, 7};
```

```
    int n = sizeof(arr) / sizeof(arr[0]);
```

```
    mergeSort(arr, 0, n - 1);
```

```
    printf("Sorted array: \n");
```

```
    for (int i = 0; i < n; i++)
```

```
        printf("%d ", arr[i]);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

Output :

```
C:\Users\blogw\OneDrive\De x + v
Sorted array:
-11 -4 5 6 7 11 12 13 180 280
Process returned 0 (0x0)   execution time : 0.910 s
Press any key to continue.
```

7. Write a C Program to perform Linear Search.

Program :

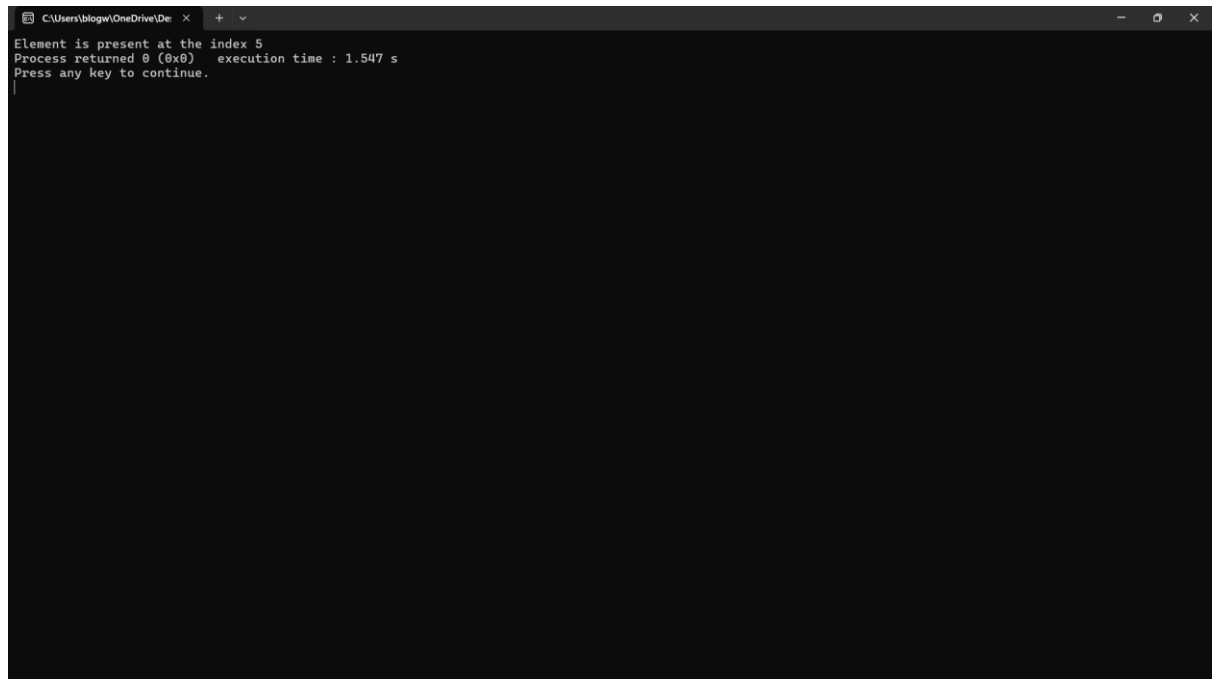
```
#include <stdio.h>
```

```
int linearSearch(int arr[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == x)
            return i;
    }
    return -1;
}
```

```
int main() {
    int arr[] = {2, 3, 4, 10, 77, -5, 12, 40};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = -5;
    int result = linearSearch(arr, n, x);
    (result == -1) ? printf("Element is not present in an array")
```

```
        : printf("Element is present at the index %d", result);  
  
    return 0;  
}
```

Output :



```
C:\Users\blogw\OneDrive\De  x  +  v  -  [icon]  x  
Element is present at the index 5  
Process returned 0 (0x0) execution time : 1.547 s  
Press any key to continue.
```

8. Write a C Program to perform Binary Search.

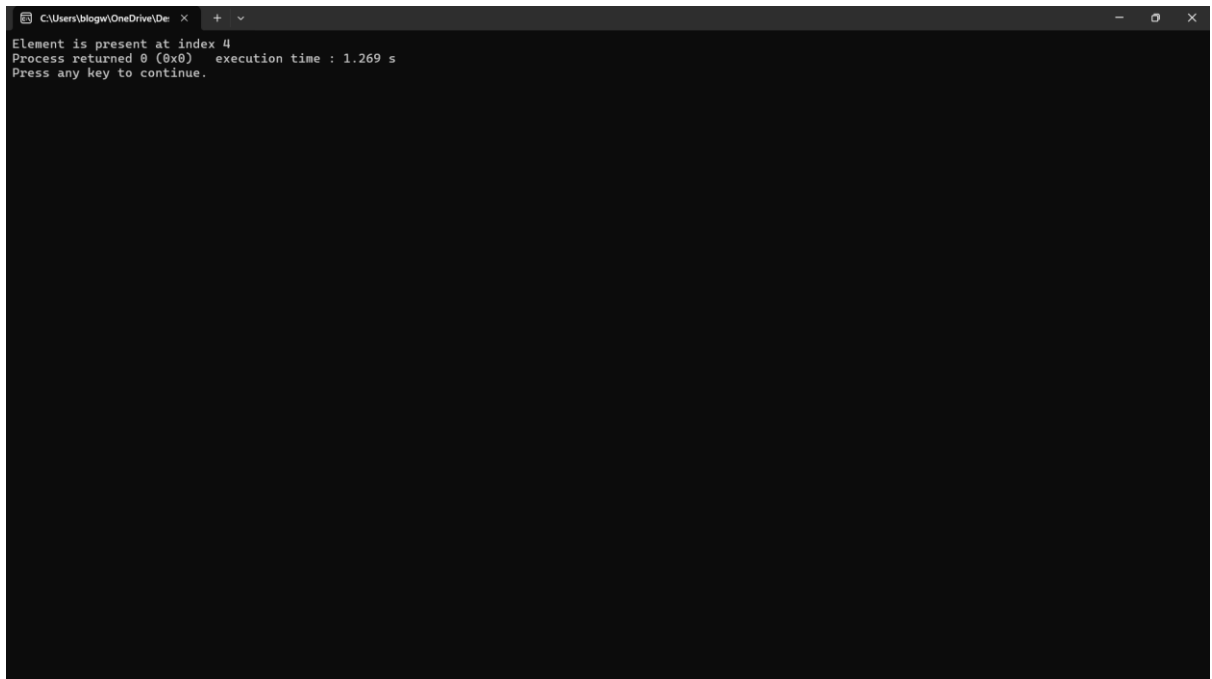
Program :

```
#include <stdio.h>
```

```
int binarySearch(int arr[], int l, int r, int x) {  
    if (r >= l) {  
        int mid = l + (r - l) / 2;  
  
        if (arr[mid] == x)  
            return mid;  
  
        if (arr[mid] > x)  
            return binarySearch(arr, l, mid - 1, x);  
        return binarySearch(arr, mid + 1, r, x);  
    }  
  
    return -1;  
}
```

```
int main() {  
    int arr[] = {2, 3, 4, 10, 40};  
    int n = sizeof(arr) / sizeof(arr[0]);  
    int x = 4;  
    int result = binarySearch(arr, 0, n - 1, x);  
    (result == -1) ? printf("Element is not present in array")  
                  : printf("Element is present at index %d", result);  
    return 0;  
}
```

Output :

A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\blogw\OneDrive\De" and standard window controls. The command prompt displays the following text: "Element is present at index 4", "Process returned 0 (0x0) execution time : 1.269 s", and "Press any key to continue." The rest of the window is black.

```
C:\Users\blogw\OneDrive\De >  
Element is present at index 4  
Process returned 0 (0x0) execution time : 1.269 s  
Press any key to continue.
```